Agent Phantom Recovery

15-rag-architecture.md

Production RAG Architecture

Version: 1.0

Status: Production Design

---

1. Purpose

This document defines the Retrieval-Augmented Generation (RAG) architecture for Agent Phantom Recovery.

The purpose of the RAG system is not simply document retrieval.

Its purpose is to provide:

- Repository understanding
- Long-context memory
- Semantic code search
- Architectural reasoning
- Evidence retrieval
- Cross-session knowledge persistence
- Long-horizon task support

for the Planner, Reasoner, and Verifier layers.

---

2. Why RAG Exists

Large language models have limitations.

They cannot reliably process:

```
Entire Repositories
Millions of Lines of Code
Months of Task History
Large Documentation Collections
Long-Term Memory
```

A retrieval system is therefore mandatory.

Without RAG

```
Repository
     ↓
LLM
```

Problems:

- Context overflow
- High token costs
- Information loss
- Poor scalability

With RAG

```
Repository
      ↓
Indexer
      ↓
Embeddings
      ↓
Vector Search
      ↓
Relevant Context
      ↓
LLM
```

Only relevant information is sent.

---

3. Core Philosophy

The RAG system exists to answer:

```
What information does the model need
right now?
```

Not:

```
Send everything.
```

---

4. RAG Architecture Overview

```
Repositories
Memory
Logs
Reports
Documentation
Browser Data
Terminal Data
        │
        ▼
 Ingestion Pipeline
        │
        ▼
 Chunking Pipeline
        │
        ▼
 Embedding Pipeline
        │
        ▼
 Vector Storage
        │
        ▼
 Retrieval Engine
        │
        ▼
 Context Builder
        │
        ▼
 Planner
 Reasoner
 Verifier
```

---

5. Knowledge Sources

The retrieval system indexes multiple knowledge domains.

Repository Knowledge

Contains:

```
Source Code
Configurations
Dependencies
```

```
Tests
Documentation
```

Memory Knowledge

Contains:

```
Project Memory
Working Memory
Experience Memory
Session Memory
```

Task Knowledge

Contains:

```
Previous Plans
Findings
Patches
Reports
```

Execution Knowledge

Contains:

```
Logs
Tool Outputs
Terminal Sessions
Browser Observations
```

---

6. RAG Layers

The architecture is divided into four layers.

Layer 1

Knowledge Ingestion

Layer 2

Knowledge Storage

Layer 3

Retrieval Engine

Layer 4

Context Construction

---

7. Knowledge Ingestion Layer

Purpose

Convert raw information into searchable knowledge.

Inputs

```
Repository
PDF
Markdown
Logs
Terminal Output
Browser Data
Reports
```

Output

```
Normalized Knowledge Objects
```

---

8. Repository Ingestion Pipeline

```
Repository
     ↓
Clone
     ↓
Parse
     ↓
Extract Metadata
     ↓
Chunk
     ↓
Embed
     ↓
Store
```

Extracted Data

```
Files
Functions
Classes
Imports
Dependencies
Comments
Documentation
```

---

9. Document Ingestion Pipeline

Supports:

```
Markdown
PDF
TXT
JSON
YAML
HTML
```

Flow

```
Document
 ↓
Parse
 ↓
Clean
 ↓
Chunk
 ↓
Embed
 ↓
Store
```

---

10. Log Ingestion Pipeline

Supports:

```
Terminal Logs
Execution Logs
Application Logs
```

Flow

```
Log
 ↓
Normalize
 ↓
Chunk
 ↓
Embed
 ↓
Store
```

---

11. Browser Ingestion Pipeline

Supports:

```
DOM Snapshots
Observations
Screenshots Metadata
```

Purpose

Provides web context for agents.

---

12. Chunking Architecture

Chunking is one of the most important components.

Poor chunking produces poor retrieval.

---

13. Repository Chunking

Never chunk blindly.

Bad:

```
Every 1000 Tokens
```

Good:

```
Function Boundaries
Class Boundaries
Module Boundaries
```

Example

```
AuthenticationService
```

should remain intact.

Do not split across chunks.

---

14. Documentation Chunking

Chunk by:

```
Headings
Sections
Subsections
```

---

15. Log Chunking

Chunk by:

```
Execution Blocks
Error Groups
Stack Traces
```

---

16. Embedding Architecture

Purpose

Convert knowledge into vector representations.

Flow

```
Chunk
 ↓
Embedding Model
 ↓
Vector
 ↓
Storage
```

---

17. Embedding Types

Repository Embeddings

```
Functions
Classes
Modules
```

Documentation Embeddings

```
Technical Docs
Reports
Specifications
```

Memory Embeddings

```
Experiences
Strategies
Architecture Knowledge
```

---

18. Vector Storage Architecture

Recommended MVP:

```
Supabase pgvector
```

Production Scale:

```
Qdrant
```

Enterprise Scale:

```
Dedicated Vector Cluster
```

19. Vector Collections

Repository Collection

Stores:

```
Code Chunks
Module Summaries
```

Memory Collection

Stores:

```
Long-Term Knowledge
```

Task Collection

Stores:

```
Plans
Findings
Reports
```

Execution Collection

Stores:

```
Logs
Observations
Evidence
```

20. Retrieval Engine

The retrieval engine determines what context is sent.

Pipeline

```
Query
 ↓
Vector Search
 ↓
Ranking
 ↓
Filtering
 ↓
Top Results
 ↓
Context Builder
```

---

21. Multi-Stage Retrieval

Single retrieval is often insufficient.

Stage 1

Broad Search

```
Top 50
```

Stage 2

Reranking

```
Top 20
```

Stage 3

Final Selection

```
Top 5-10
```

---

22. Retrieval Ranking

Ranking should consider:

```
Semantic Similarity
Importance
Recency
Task Relevance
Verification Score
```

Ranking Formula

```
Similarity
+
Importance
+
Recency
+
Task Weight
```

---

23. Planner Retrieval Strategy

Planner retrieves:

```
Goals
Plans
Task History
Project Memory
```

Planner Context Size

Small.

Typically:

```
5-15 Chunks
```

---

24. Reasoner Retrieval Strategy

Reasoner retrieves:

```
Code
Dependencies
Architecture
Logs
Evidence
```

Reasoner Context Size

Largest context allocation.

Typically:

```
20-50 Chunks
```

---

25. Verifier Retrieval Strategy

Verifier retrieves:

```
Patch
Evidence
Tests
Reports
```

Goal

Challenge conclusions.

Not repeat analysis.

---

26. Context Builder

Purpose

Convert retrieved knowledge into model-ready context.

Pipeline

```
Retrieved Chunks
 ↓
Deduplicate
 ↓
Rank
 ↓
Compress
 ↓
Structure
 ↓
LLM Context
```

---

27. Context Compression

Avoid sending raw retrieval results.

Instead:

```
Retrieve
 ↓
Summarize
 ↓
Compress
 ↓
Send
```

Benefits

```
Lower Tokens
Higher Signal
Better Accuracy
```

---

28. Repository Intelligence Integration

RAG works closely with repository intelligence.

Additional Inputs

```
Call Graph
Dependency Graph
Architecture Graph
```

Benefit

Better code understanding.

---

29. Memory Integration

Memory is implemented on top of RAG.

Memory Flow

```
Experience
 ↓
Store
 ↓
Embed
 ↓
Retrieve
 ↓
Reuse
```

---

30. Experience Memory

Stores:

```
Successful Fixes
Failed Fixes
Strategies
Lessons Learned
```

Example

```
Authentication Bug
 ↓
Fix Successful
 ↓
Store Pattern
```

Future tasks can reuse it.

---

31. Long-Horizon Task Support

Tasks may last:

```
Hours
Days
```

Conversation history alone is insufficient.

Solution

```
State
+
Memory
+
RAG
```

---

32. Evidence Retrieval

Evidence is a first-class retrieval target.

Evidence Types

```
Logs
Screenshots
Terminal Output
Code
Reports
```

Importance

All major conclusions should reference evidence.

---

33. Verification Retrieval

Verifier requires separate retrieval.

Retrieve

```
Evidence
Patch
Tests
Regression Data
```

Goal

Independent review.

---

34. Antigravity IDE Integration

RAG powers multiple workspaces.

Source Code Workspace

Provides:

```
Semantic Search
Reference Search
Architecture Retrieval
```

Memory Workspace

Provides:

```
Knowledge Retrieval
Experience Retrieval
```

Timeline Workspace

Provides:

```
Historical Context
```

Agent Workspace

Provides:

```
Task Context
```

---

35. Performance Targets

Repository Retrieval:

```
< 1 Second
```

Memory Retrieval:

```
< 500ms
```

Reranking:

```
< 1 Second
```

Context Construction:

```
< 2 Seconds
```

---

36. Security Requirements

Never embed:

```
Secrets
Passwords
Tokens
Credentials
```

Preprocessing

```
Detect Secret
 ↓
Mask
 ↓
Embed
```

---

37. Observability

Track:

```
Retrieval Latency
Recall
Precision
Chunk Usage
Context Size
```

Metrics

```
Hit Rate
Retrieval Success
Ranking Quality
```

---

38. Future Expansion

Future additions:

```
Knowledge Graph
Code Graph Retrieval
Hybrid Search
Agent Memory Networks
```

Hybrid Retrieval

Combine:

```
Vector Search
+
Keyword Search
+
Graph Search
```

---

39. Recommended Production Stack

MVP

```
Supabase pgvector
```

Production

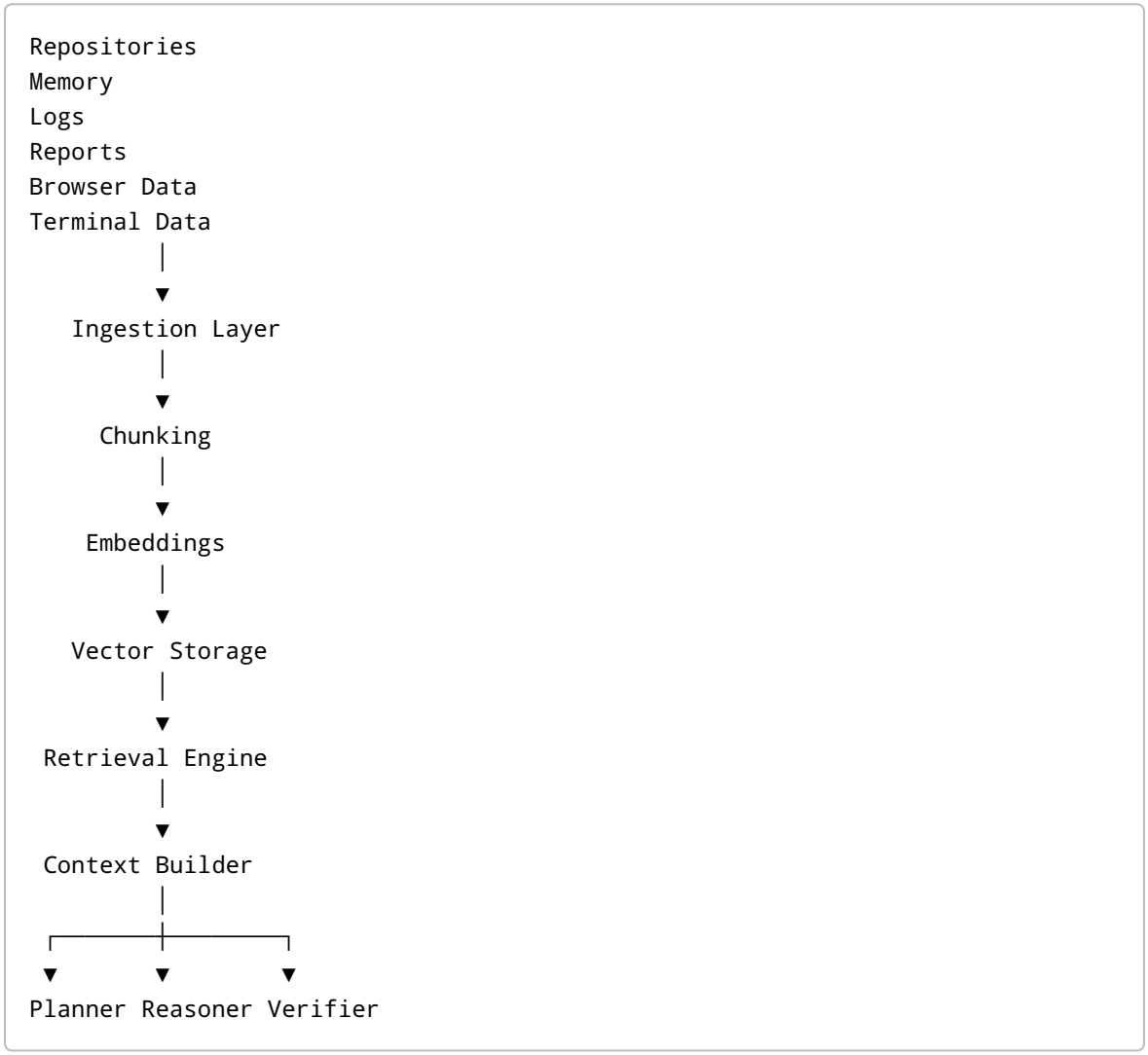
```
Qdrant
+
PostgreSQL
+
Redis
```

Enterprise

```
Qdrant Cluster
+
Knowledge Graph
+
Dedicated Retrieval Service
```

---

40. Final RAG Architecture



The RAG system serves as the knowledge backbone of Agent Phantom Recovery, enabling scalable repository understanding, long-context memory, semantic retrieval, evidence-driven reasoning, and long-horizon autonomous engineering workflows inside Antigravity IDE.

End of Document

Status: Approved

Version: 1.0

Document Type: Production RAG Architecture